Sistemas de Recomendación



Arias, Blanc, Comelli

Sistemas de recomendación

W. Adrián Arias Nicolás Blanc Jerónimo Comelli

2026

Índice general

0.1. Introducción 1
0.2. Preliminares y motivación 1

1. Conceptos latentes **3**
1.1. Gradiente Descendiente 4
1.1.1. Gradiente descendiente: búsqueda de direcciones para métodos de búsqueda lineal 5
1.2. Gradiente Descendiente Estocástico 6
1.3. Regularización 6
1.4. Gradiente Descendiente con Regularización 7
1.5. Gradiente Descendiente Estocástico con Regularización 7
1.6. Gradiente Descendente y Descendente estocástico. 8

0.1. Introducción

No es novedad que hoy los sistemas de recomendación son parte de nuestra vida cotidiana. En las redes sociales, en la publicidad segmentada e incluso en las plataformas de Streaming.

Esto no es para nada grave, quien no ha visto una película recomendada por una plataforma que resultó ser muy buena? En general los sistemas de recomendación suelen ser muy útiles tanto para los usuarios que reciben el contenido que les interesa como para los proveedores que pueden dirgir sus productos a un público en particular.

Los sistemas de recomendación son muy utilizados y francamente exitosos en el mundo comercial pero, ¿podremos ampliar su uso con un enfoque macroeconómico? Supongamos por un momento que somos un país productor y tenemos países que son habituales consumidores de nuestras exportaciones.

Naturalmente, uno buscaría mantener la competitividad colocando nuevos productos o ampliando la cartera de países importadores, pero esto puede ser muy costoso y además, en algunos casos, no está tan claro que países son potenciales clientes.

Es en este punto donde los **sistemas de recomendación** capturando la información necesaria, pueden ayudar a predecir países interesados en nuestros productos. En particular, se discutirán y se expondrán modelos de **factores latentes**

0.2. Preliminares y motivación

Dada una matriz $R \in \mathbb{R}^{n\times m}$, no siempre será conveniente, o incluso posible, calcular una factorización exacta de la misma. Resulta natural, entonces, buscar una aproximación de rango reducido.

Es decir, buscamos matrices U y V tales que

$$R \approx UV^T.$$

Para determinar qué tan buena es nuestra aproximación, podemos hacer uso de la norma de Frobenius. En consecuencia, buscamos U y V de modo que la diferencia entre R y su aproximación sea tan pequeña como sea posible, es decir, que

$$\|R - UV^T\|_F$$

sea mínima. Podemos formular este problema como un problema de optimización, en el cual buscamos las matrices U y V que minimicen el error de aproximación:

$$\min_{U,V} J(U,V) = \|R - UV^T\|_F.$$

Dependiendo de la naturaleza de las entradas de la matriz R y de las propiedades que deseemos imponer sobre la factorización, podemos requerir que U y V satisfagan determinadas condiciones. Por ejemplo, si buscamos una factorización basada en matrices ortogonales y valores singulares, podemos utilizar la *descomposición en valores singulares* (*Singular Value Decomposition*, SVD).

Sin embargo, en el problema que nos interesa, la matriz R no contiene simplemente números sin significado. Sus entradas representan información acerca de algún fenómeno o conjunto de objetos de interés. Por ejemplo, podemos considerar una matriz cuyas filas representan usuarios de una plataforma de *streaming* y cuyas columnas representan películas. En este caso, la entrada r_{ij} puede representar la valoración del usuario i a la película j.

Nuestro objetivo, sin embargo, no consiste simplemente en aproximar una matriz R completamente conocida. Si todas las entradas de R fueran conocidas, no habría necesidad de predecirlas. El problema surge precisamente porque, en general, solo conocemos una pequeña parte de sus entradas.

Un usuario puede haber visto diez, veinte o incluso cien películas, mientras que una plataforma puede disponer de miles o millones de títulos. Por lo tanto, en la matriz de valoraciones habrá relativamente pocas entradas conocidas, correspondientes a las películas que el usuario ha visto y valorado y, una gran cantidad de entradas desconocidas, correspondientes a películas que todavía no ha visto.

El objetivo de un sistema de recomendación consiste, entonces, en utilizar la información disponible para inferir los valores de las entradas desconocidas. En nuestro ejemplo, buscamos determinar qué películas que el usuario aún no ha visto podrían resultar de su interés.

Para abordar este problema, debemos encontrar una forma de completar las entradas desconocidas de manera coherente con la información disponible. No se trata simplemente de asignar valores arbitrarios a las entradas faltantes, sino de construir una aproximación que preserve, en la medida de lo posible, la estructura subyacente de los datos observados.

Este problema constituye una instancia de lo que se conoce como *completado de matrices* (*matrix completion*), y será el punto de partida para nuestro estudio.

Para comenzar este recorrido por los sistemas de recomendación, estudiaremos primero un método sencillo que, aunque puede resultar computacionalmente costoso, permite obtener resultados satisfactorios y sirve como punto de partida para métodos más sofisticados.

Capítulo 1

Conceptos latentes

En muchos problemas de aprendizaje automático, los datos pueden presentar características subyacentes que no son directamente observables, pero que permiten explicar las relaciones entre las distintas observaciones. La identificación de estas características puede resultar de gran utilidad, ya que permite modelar tanto las preferencias de los usuarios como las propiedades de los elementos con los que estos interactúan. A estas características no observadas las denominaremos *conceptos latentes*.

Consideremos una matriz de datos $R \in R^{m \times n}$, donde la entrada r_{ij} representa la valoración del usuario i sobre el *ítem* j. Una forma de identificar estructuras latentes en los datos consiste en aproximar R mediante una factorización matricial de rango bajo,

$$R \approx UV^T,$$

1 donde $U \in \mathbb{R}^{m \times k}$ y $V \in \mathbb{R}^{n \times k}$, con $k \ll \min(m, n)$.

Denotemos por $\vec{u}_i$ a la fila i de U y por $\vec{v}_j$ a la fila j de V. Entonces, la entrada (i, j) de UV^T está dada por

$$(UV^T)_{ij} = \vec{u}_i \cdot \vec{v}_j,$$

por lo que podemos interpretar la predicción de la valoración del usuario i sobre el ítem j como

$$\hat{r}_{ij} = \vec{u}_i \cdot \vec{v}_j.$$

Cada uno de los vectores $\vec{u}_i, \vec{v}_j \in \mathbb{R}^k$ constituye una representación de dimensión reducida de un usuario y de un ítem, respectivamente. En particular, podemos interpretar cada una de sus coordenadas como una medida de la relación de dicho usuario o ítem con una determinada dimensión latente. Estas dimensiones no tienen necesariamente una interpretación semántica explícita; su significado emerge de la estructura de los datos y del proceso de factorización.

Llamaremos **factores latentes** a los vectores $\vec{u}_i$ y $\vec{v}_j$. De esta manera, el producto interno entre ambos vectores determina la valoración que el modelo asigna al usuario i para el ítem j.

El parámetro k determina la dimensión del espacio latente. Si k es menor que el rango de R, no buscamos obtener una factorización exacta, sino una aproximación de rango reducido,

$$R \approx UV^T.$$

La calidad de esta aproximación puede medirse mediante la norma de Frobenius,

$$\|R - UV^T\|_F.$$

En consecuencia, la búsqueda de una aproximación de rango bajo puede plantearse como un problema de optimización, en el que buscamos determinar U y V de modo que minimicen el error de reconstrucción. En el caso de disponer de la matriz completa, esto conduce al problema

$$\min_{U,V} \frac{1}{2}\|R - UV^T\|_F^2.$$

Sin embargo, en nuestro problema no disponemos de todas las entradas de la matriz R. El objetivo es precisamente utilizar las valoraciones conocidas para predecir aquellas que faltan. Por este motivo, no resulta apropiado calcular directamente la norma de Frobenius sobre toda la matriz, ya que esta requiere conocer todas sus entradas.

Para abordar esta situación, consideremos el conjunto de índices correspondientes a las entradas observadas,

$$S = \{(i, j) : r_{ij} \text{ es conocido}\}.$$

A partir de estas entradas, buscamos determinar una factorización de rango bajo UV^T que permita reconstruir adecuadamente los valores observados y, posteriormente, utilizarla para predecir los valores faltantes.

En particular, para una entrada $(i, j) \notin S$, la valoración predicha por el modelo está dada por

$$\hat{r}_{ij} = \vec{u}_i \cdot \vec{v}_j.$$

Para las entradas observadas, definimos el error de reconstrucción como

$$e_{ij} = r_{ij} - \hat{r}_{ij} = r_{ij} - \vec{u}_i \cdot \vec{v}_j.$$

Por lo tanto, podemos formular el problema de encontrar los factores latentes como la minimización del error cuadrático sobre las entradas observadas:

$$\min_{U,V} \frac{1}{2} \sum_{(i,j)\in S} e_{ij}^2.$$

O, equivalentemente,

$$\min_{U,V} \frac{1}{2} \sum_{(i,j)\in S} \left(r_{ij} - \vec{u}_i \cdot \vec{v}_j\right)^2.$$

Esta formulación permite determinar los factores latentes utilizando únicamente la información disponible en las entradas observadas, para posteriormente emplearlos en la predicción de las valoraciones faltantes.

1.1. Gradiente Descendiente

El conjunto de datos de validación es un subconjunto de los datos disponibles para el entrenamiento del modelo. Un problema práctico con este enfoque es que la cantidad de datos disponibles es limitada e, idealmente, queremos utilizar la mayor cantidad posible de ellos para entrenar el modelo. Esto requiere que nuestro conjunto de validación V sea relativamente pequeño, lo que puede llevar a una estimación ruidosa (de alta varianza) de la capacidad de predicción del modelo.

Una solución para estos objetivos contrapuestos —contar con un conjunto de entrenamiento grande y, al mismo tiempo, con un conjunto de validación suficientemente grande— es utilizar la validación cruzada. La validación cruzada K-fold particiona el conjunto de datos en K partes. En cada iteración, $K-1$ de estas partes forman el conjunto de entrenamiento R, mientras que la parte restante constituye el conjunto de validación V.

La validación cruzada itera sobre las distintas elecciones posibles del conjunto de validación. Este proceso se repite para cada elección de V, entrenando un nuevo modelo en el conjunto complementario R. Finalmente, el rendimiento del modelo se obtiene promediando los resultados de las K corridas. En particular, particionamos el conjunto de datos como

$$D = R \cup V, \quad \text{tal que} \quad R \cap V = \emptyset.$$

Donde V es el conjunto de validación. Entrenamos el modelo utilizando los datos de R y, posteriormente, evaluamos el rendimiento del predictor f sobre V, utilizando alguna medida de error, por ejemplo, el error cuadrático.

Más precisamente, para cada partición k, se produce un predictor $f^{(k)}$, el cual es aplicado al conjunto de validación $V^{(k)}$ para calcular el riesgo empírico

$$R(f^{(k)}, V^{(k)}).$$

Iteramos sobre las distintas formas de elegir los conjuntos de entrenamiento y validación, y calculamos la media de los errores obtenidos.

La validación cruzada aproxima el error de generalización esperado mediante

[Insertar gráfico.]

Una desventaja potencial de la validación K-fold es el costo computacional asociado con entrenar el modelo K veces, lo cual puede resultar muy costoso. Además, en la práctica, generalmente no es suficiente fijarse directamente en los parámetros del modelo de manera aislada.

Necesitamos explorar múltiples parámetros complejos, como pueden ser múltiples parámetros de regularización, que incluso pueden no formar parte directamente de los parámetros del modelo. Evaluar la calidad del modelo dependiendo de estos hiperparámetros puede resultar en un número de corridas que crece exponencialmente con el número de parámetros.

Una posibilidad es utilizar validación cruzada anidada para buscar buenos hiperparámetros. Sin embargo, la validación cruzada es un problema embarazosamente paralelo, es decir, resulta sencillo separar el problema en muchos problemas pequeños e independientes. Dados suficientes recursos computacionales, la validación cruzada puede realizarse de manera paralela, reduciendo considerablemente el tiempo total de evaluación.

[Insertar dibujo.]

1.1.1. Gradiente descendiente: búsqueda de direcciones para métodos de búsqueda lineal

Nocedal, pág. 21.

La dirección de máximo crecimiento ∇f_k sugiere de manera natural que la dirección opuesta,

$$-\nabla f_k,$$

es una elección apropiada para buscar direcciones en un método de búsqueda lineal. Intuitivamente, de todas las direcciones posibles, esta es la dirección en la que la función decrece más rápidamente.

Para verificar esta observación, apelamos al teorema de Taylor, que nos dice que, para toda dirección de búsqueda p y un paso α suficientemente pequeño,

[Insertar gráfico.]

La tasa de cambio de f_k en la dirección p en el punto x_k está determinada por el coeficiente de α, dado por

$$p^T \nabla f_k.$$

Por lo tanto, la dirección unitaria de mayor decrecimiento es la solución del problema

$$\min_p \; p^T \nabla f_k \qquad \text{sujeto a} \qquad \|p\| = 1.$$

Dado que

$$p^T \nabla f_k = \|p\| \, \|\nabla f_k\| \cos\theta,$$

donde θ es el ángulo entre p y ∇f_k, y puesto que p es unitario, tenemos

$$p^T \nabla f_k = \|\nabla f_k\| \cos\theta.$$

Esta expresión alcanza su menor valor cuando $\theta = \pi$, es decir, cuando

$$p = -\frac{\nabla f_k}{\|\nabla f_k\|}.$$

[Insertar gráfico.]

El método de máximo decrecimiento es un método de búsqueda lineal que se mueve en la dirección

$$p_k = -\nabla f_k$$

en cada paso. El tamaño del paso puede seleccionarse de varias maneras. Una ventaja de este método es que requiere únicamente el cálculo del gradiente y no de las segundas derivadas.

Sin embargo, este método puede ser muy lento en problemas difíciles. Los métodos de búsqueda lineal pueden buscar en otras direcciones distintas de la dirección de máximo decrecimiento. En general, cualquier dirección de decrecimiento que forme un ángulo menor que $\pi/2$ con respecto a $-\nabla f_k$ garantiza una mejora de la función objetivo para un tamaño de paso suficientemente pequeño.

[Insertar grafico.]

1.2. Gradiente Descendiente Estocástico

1.3. Regularización

Uno de los principales problemas a la hora de trabajar con matrices dispersas es la escasez de información disponible para realizar las predicciones.

Puede ocurrir, en el caso de matrices dispersas, que nuestra aproximación sea teóricamente buena, en el sentido de que el error asociado al problema de minimización sea muy pequeño, pero que, en la práctica, las predicciones correspondientes a los valores no observados no sean acordes con lo que esperaríamos.

Por ejemplo, si nos encontramos en el contexto de una matriz de calificaciones de usuarios sobre películas, podría resultar poco razonable que el modelo recomiende una película de terror a un usuario cuyas preferencias observadas muestran una clara inclinación por las películas de comedia.

Para mitigar estos efectos, haremos uso de la *regularización de Tikhonov*. Esencialmente, buscamos controlar mediante un factor $\lambda > 0$ el tamaño de los parámetros que determinan las predicciones, favoreciendo soluciones más simples y evitando que el modelo se ajuste excesivamente a los datos observados.

[Agregar gráfico comparativo entre la distribución de las predicciones con y sin regularización.]

Matemáticamente, incorporamos un término de regularización al problema de minimización. En nuestro caso, utilizaremos una penalización cuadrática sobre los factores que intervienen en la factorización de la matriz. De esta manera, el problema queda dado por

$$\min J = \frac{1}{2} \sum_{(i,j)\in S} e_{ij}^2 + \frac{\lambda}{2} \sum_{i,s} u_{is}^2 + \frac{\lambda}{2} \sum_{j,s} v_{js}^2.$$

El primer término corresponde al error sobre los valores observados, mientras que los dos términos restantes constituyen la penalización de Tikhonov sobre los factores U y V.

El parámetro λ controla el compromiso entre ambos objetivos. Para valores pequeños de λ, el modelo tiene mayor libertad para ajustarse a los datos observados, mientras que valores grandes de λ penalizan con mayor intensidad los parámetros de gran magnitud y favorecen soluciones más restringidas.

1.4. Gradiente Descendiente con Regularización

1.5. Gradiente Descendiente Estocástico con Regularización

1.6. Gradiente Descendente y Descendente estocástico.

Tabla 1.1: Convergence and Performance Comparison ($K = 2$ vs. $K = 8$)

Algorithm	K	**Iterations**	**Final RMSE**	**RMSE Change (Δ)**	**Status**
Batch GD	2	647	1,2855	$3{,}7 \times 10^{-5}$	Converged
SGD (Aggarwal 2016)	2	65	0,7445	$1{,}0 \times 10^{-5}$	Converged
Batch GD (w/ reg.)	2	1000	1,2442	—	Max Iterations
SGD (w/ reg.)	2	39	0,8009	$1{,}3 \times 10^{-5}$	Converged
Batch GD	8	1000	0,6614	—	Max Iterations
SGD (Aggarwal 2016)	8	56	0,5905	$1{,}7 \times 10^{-5}$	Converged
Batch GD (w/ reg.)	8	1000	0,6691	—	Max Iterations
SGD (w/ reg.)	8	100	0,6509	$5{,}5 \times 10^{-5}$	Converged

Tabla 1.2: Esquema del experimento para 10-fold cross-validation.

Parametros	Valor
Tamaño de la matriz	610×9724
Valores observados	100 836 (1.70 % dense)
Número de folds	10
Factores elegidos k	2, 5, 8, 10, 20, 30, 50
λ	0.01
γ	0.01
Iteraciones maximas	1000
Tolerancia	10^{-4}

Tabla 1.3: Resultados de la validación cruzada, listados por RMSE. The generalization gap is the mean test RMSE minus the mean train RMSE.

k	λ	Test RMSE	SD	Train RMSE	SD	Iteraciones	Convergencia	Tiempo
2	0.01	1.0893	0.0100	0.7463	0.3431	47	10/10	6m 33.1s
5	0.01	1.1060	0.0129	0.6393	0.4667	60	10/10	8m 24.1s
8	0.01	1.1367	0.0125	0.5680	0.5688	67	10/10	9m 50.7s
10	0.01	1.1451	0.0123	0.5278	0.6173	68	10/10	9m 52.3s
50	0.01	1.1891	0.0117	0.1545	1.0346	108	10/10	16m 51.4s
20	0.01	1.1907	0.0182	0.3726	0.8181	85	10/10	12m 21.3s
30	0.01	1.1999	0.0096	0.2748	0.9250	90	10/10	13m 39.5s

Tabla 1.4: Test RMSE per fold for each number of factors k ($\lambda = 0{,}01$).

	Number of factors k						
Fold	2	5	8	10	20	30	50
1	1.1014	1.1204	1.1484	1.1624	1.2143	1.2176	1.2005
2	1.0882	1.1042	1.1412	1.1379	1.1826	1.2004	1.1906
3	1.1006	1.1238	1.1562	1.1561	1.2061	1.2070	1.2019
4	1.0831	1.1003	1.1283	1.1314	1.1743	1.1944	1.1856
5	1.1018	1.1163	1.1509	1.1655	1.2169	1.2050	1.2103
6	1.0729	1.0895	1.1178	1.1323	1.1624	1.1890	1.1728
7	1.0861	1.0957	1.1303	1.1350	1.1798	1.1994	1.1869
8	1.0794	1.0869	1.1311	1.1420	1.1775	1.1981	1.1820
9	1.0849	1.1081	1.1244	1.1447	1.1989	1.1836	1.1783
10	1.0948	1.1148	1.1387	1.1435	1.1939	1.2037	1.1825
Mean	1.0893	1.1060	1.1367	1.1451	1.1907	1.1999	1.1891
SD	0.0100	0.0129	0.0125	0.0123	0.0182	0.0096	0.0117

Tabla 1.5: Final train RMSE per fold for each number of factors k ($\lambda = 0{,}01$). The initial train RMSE was approximately 3.653 in every fold.

	Number of factors k						
Fold	2	5	8	10	20	30	50
1	0.7418	0.6375	0.5636	0.5297	0.3588	0.2688	0.1475
2	0.7538	0.6376	0.5646	0.5280	0.3752	0.2739	0.1485
3	0.7485	0.6336	0.5645	0.5216	0.3694	0.2746	0.1531
4	0.7399	0.6412	0.5660	0.5325	0.3829	0.2801	0.1484
5	0.7416	0.6362	0.5621	0.5228	0.3607	0.2771	0.1529
6	0.7532	0.6369	0.5747	0.5325	0.3808	0.2686	0.1603
7	0.7525	0.6464	0.5717	0.5269	0.3876	0.2818	0.1571
8	0.7409	0.6595	0.5680	0.5211	0.3741	0.2702	0.1570
9	0.7560	0.6316	0.5742	0.5284	0.3643	0.2928	0.1464
10	0.7344	0.6325	0.5699	0.5340	0.3720	0.2601	0.1742
Mean	0.7463	0.6393	0.5680	0.5278	0.3726	0.2748	0.1545

Tabla 1.6: Epochs until convergence per fold for each number of factors k ($\lambda = 0{,}01$). All 70 fits converged.

	Number of factors k						
Fold	2	5	8	10	20	30	50
1	55	60	75	65	111	99	124
2	36	60	72	71	81	89	119
3	41	70	69	75	85	90	107
4	49	53	68	60	70	81	121
5	55	61	74	71	103	85	109
6	37	64	60	63	75	100	97
7	36	48	62	67	65	81	101
8	55	38	69	80	81	94	102
9	35	73	59	69	98	70	125
10	67	72	66	64	84	113	80
Mean	47	60	67	68	85	90	108
Min	35	38	59	60	65	70	80
Max	67	73	75	80	111	113	125

Tabla 1.7: Running time per fold in seconds for each number of factors k ($\lambda = 0{,}01$). The last row gives the total time per combination.

	Number of factors k						
Fold	2	5	8	10	20	30	50
1	45.6	50.2	61.9	54.4	99.2	92.2	123.0
2	29.3	50.2	63.6	63.6	72.1	81.1	111.6
3	38.0	58.5	65.4	65.2	71.2	81.5	100.5
4	41.4	44.2	62.3	50.4	58.1	73.7	113.6
5	46.5	50.4	63.2	59.7	87.0	76.7	100.1
6	30.9	53.4	54.6	53.8	62.2	90.6	87.3
7	30.0	40.6	53.5	61.8	55.1	73.1	88.0
8	46.5	31.9	59.2	69.4	71.9	84.6	97.3
9	29.0	63.6	50.8	59.0	85.9	63.7	115.4
10	55.8	61.0	55.9	54.8	78.6	102.1	74.4
Total	6m 33.1s	8m 24.1s	9m 50.7s	9m 52.3s	12m 21.3s	13m 39.5s	16m 51.4s

Tabla 1.8: SGD refit on all 100 836 observed ratings with the best combination ($k = 2$, $\lambda = 0{,}01$, $\gamma = 0{,}01$). Total drop in RMSE: 2.9080 (79.6 %); run time 52.7 s.

Iteration	Train RMSE	Change in RMSE
0 (init)	3.6535	–
10	0.8364	0.014911
20	0.7779	0.003289
30	0.7613	0.000408
40	0.7530	0.000555
50	0.7480	0.000394
55 (converged)	0.7455	0.000073

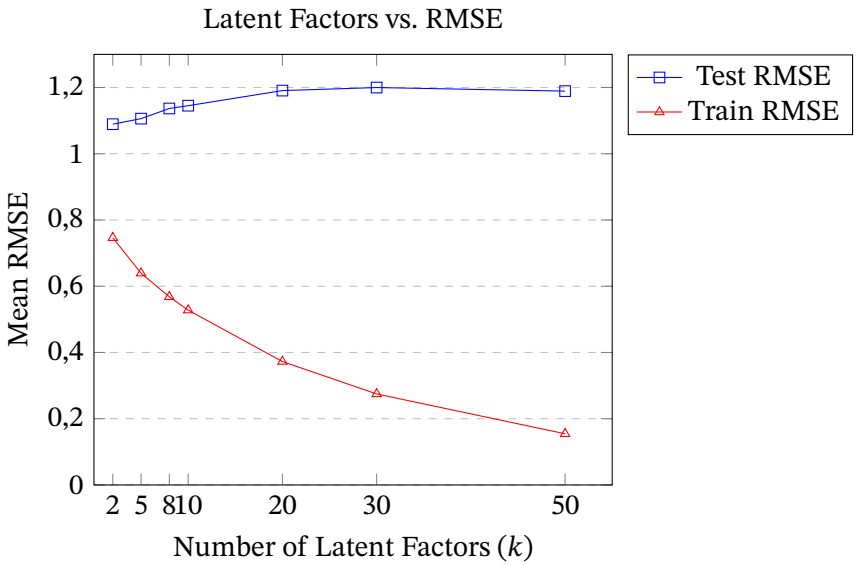


Figura 1.1: Model error and generalization gap as a function of latent factors based on results2.txt.

Bibliografía

[1] C. Aggarwal, C. Procopiuc y P. S. Yu. «Finding localized associations in marketbasket data». En: *IEEE Transactions on Knowledge and Data Engineering* 14.1 (2001), págs. 51-62.

[2] C. C. Aggarwal. *Recommender Systems: The Textbook*. Springer International Publishing, 2016. DOI: 10.1007/978-3-319-29659-3.

[3] M. P. Deisenroth, A. A. Faisal y C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[4] T. Hastie, R. Tibshirani y J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer Series in Statistics. Springer, 2009.

[5] G. James, D. Witten, T. Hastie y R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2nd. Springer, 2021.

[6] J. Nocedal y S. J. Wright. *Numerical Optimization*. Springer, 1999.